

Proiect SeniorWatch - Partea a 2-a: PROIECTARE

Foaie de titlu și semnături autorizate

Câmp	Valoare
Denumire proiect	SeniorWatch - Sistem de Teleasistență la Domiciliu a Persoanelor în Vârstă
Cod proiect	MPSAM-2026-AAL-01
Client	Fundația „Bătrânii sunt ai noștri”, Str. Sapienței nr. 28/A, Timișoara, cod 300555
Reprezentant client	Dr. Ionescu Gheorghe
Data demarării	26.03.2026
Data estimată de finalizare	30.05.2026
Versiune document	2.0

Semnături autorizate (firma de software):

Rol	Nume, prenume	Semnătura
Programator-șef	Balan Dan	
Ajutor programator-șef	Ardelean Marius	
Secretar / Manager	Balan Cristina	

Titulari disciplină (avizare academică): Prof.dr.ing. Lăcrimioara Stoicu Tivadar, Prof.dr.ing. Vasile Stoicu Tivadar, Ș.l.dr.ing. Dorin Berian.

Colectivul de elaborare și responsabilitățile de codificare

Documentația de proiectare distribuie componentele arhitecturale pe responsabili, respectând principiul „o componentă → un responsabil principal”, cu revizuire încrucișată pe granițele de integrare.

Rol în echipă	Nume, prenume	Componentă atribuită (modul)
Programator-șef	Balan Dan	Backend Cloud (src/cloud), securitate JWT, model de date, coordonare tehnică
Ajutor programator-șef	Ardelean Marius	Infrastructură AWS (Elastic Beanstalk, RDS, S3, SNS, CloudFront), deployment
Secretar / Manager	Balan Cristina	Documentație, planificare, comunicare client, avizare
Programator	Mathe Alexandra	Firmware ESP32 (src/embedded), integrare BLE / senzori
Programator	Morgovan Raluca	Interfață web (src/web) - partea medic (fișă, consultații, grafice, HL7/FHIR)
Programator	Nicola Larisa	Interfață web (src/web) - partea pacient & rapoarte
Programator	Cotolan Denisa	Aplicație mobilă Android (modul de arhitectură)

Echipa respectă metoda echipei programatorului-șef: activitatea este condusă de programatorul-șef, secondat de un ajutor care cunoaște proiectul în detaliu pentru a prelua conducerea în caz de forță majoră, iar secretarul/managerul degrevează echipa de activitățile de rutină.

Cuprins

1. Arhitectura programului

- 1.1. Descriere succintă și principii de funcționare

- 1.2. Modele arhitecturale și tehnologii
- 1.3. Mediul de dezvoltare
- 1.4. Sistemul de operare și mediul de execuție
- 1.5. Schema-bloc a arhitecturii
- 1.6. Diagrama de secvență a fluxului principal
- 1.7. Arborescența dialogurilor (formele aplicațiilor)

2. Descrierea componentelor (modulelor)

- 2.1. Șablon unitar de descriere
- 2.2. Modulul ESP32, Mobil, Cloud, Web, Tipuri partajate

3. Descrierea comunicării între module

- 3.1. Șablon unitar pentru canale
- 3.2. Canalele efective (C1-C7) + harta integrată

4. Structuri de baze de date și fișiere

- 4.1. Schema generală (diagrama ER)
- 4.2. Definiții de câmp pe tabele
- 4.3. Legături, indecși, drepturi de acces
- 4.4. Normalizarea
- 4.5. Fișiere și stocare în afara bazei de date

1. Arhitectura programului

1.1. Descriere succintă și principii de funcționare

SeniorWatch este un sistem distribuit de teleasistență medicală, structurat pe patru componente (module) fizice interconectate printr-un Cloud central. Lanțul operațional este: **senzori** → **modul inteligent (ESP32)** → **smartphone (Android)** → **Cloud (REST API pe AWS)** → **aplicație web a medicului**, completat de interoperabilitatea cu sistemul medicului de familie (HL7 FHIR).

Cele patru componente și rolurile lor:

- Modulul Inteligent (ESP32 + senzori).** Colectează semnalele de la senzorul optic de puls/oximetrie (SEN-11574, pe pin analogic), modulul de activitate cardiacă AD8232 (semnal ECG + detecție lead-off) și senzorul de temperatură/umiditate DHT; compune cadre de măsurători și le transmite prin Bluetooth Low Energy (BLE GATT). Firmware-ul colectează probe la ~1 s și emite un cadru mediat la ~10 s.
- Aplicația Mobilă (Android).** Interfața pacientului/îngrijitorului: primește cadrele BLE de la ESP32, le agregă într-un lot (batch) și le încarcă în Cloud prin REST (POST /api/measurements), funcționând offline-first. La depășirea pragurilor trimite un eveniment de alarmă (POST /api/alerts). *(Această componentă este descrisă la nivel de arhitectură.)*
- Componenta Cloud (backend).** Server central REST realizat în **Spring Boot 3.3.4 / Java 21**, desfășurat pe **AWS Elastic Beanstalk** (Docker / Amazon Linux 2023), cu persistență în **Amazon RDS PostgreSQL**. Asigură autentificarea pe roluri (JWT), evidența utilizatorilor și a pacienților, ingestia idempotentă a măsurătorilor, gestiunea recomandărilor și a alarmelor, jurnalul de audit append-only și expunerea datelor pentru export HL7/FHIR.
- Aplicația Web.** Portal **React (SPA)** pentru medic, administrator și pacient: fișă pacient, consultații ICD-10, alergii, recomandări, grafice de evoluție (inclusiv ECG), rapoarte și generarea documentelor HL7 FHIR R4. Este servit prin **Amazon CloudFront + S3**.

Principii generale de funcționare:

- Achiziție periodică + agregare:** ESP32 citește la ~1 s și emite un cadru mediat la ~10 s; aplicația Android agregă mai multe cadre într-un lot înainte de a-l trimite către Cloud - reduce traficul și consumul.
- Alarmare în timp cvasi-real:** evaluarea pragurilor se face local pe mobil, iar la depășire se trimite imediat și asincron un eveniment de alarmă (POST /api/alerts).
- Offline-first cu idempotență:** în lipsa internetului, loturile și alarmele se stochează local și se retrimite la reconectare; deduplicarea pe cheie unică (batchId pentru loturi, id UUID pentru alarme) împiedică dublurile.
- Securitate „by design”:** comunicare TLS, autorizare pe roluri (JWT Bearer), izolarea datelor per medic (filtrare pe doctor_id), parole bcrypt, jurnal de audit append-only - conform GDPR și criteriilor de calitate EuroRec Seal Level 2.
- Interoperabilitate standardizată:** emitere HL7 **FHIR R4** (Bundle XML) generat din portalul web; schema bazei de date prevede stocarea trimerilor HL7 inbound și a scrisorilor FHIR outbound.

Platforma Cloud de referință. Componenta Cloud este realizată pe **Amazon Web Services (AWS)**, regiunea eu-central-1 (Frankfurt). Lanțul de participanți este **Pacient** → **Modul ESP32** → **App Android** → **REST API (Spring Boot, Elastic Beanstalk)** → **RDS PostgreSQL**, completat de aplicația Web (portal medic/pacient) și de sistemul extern al medicului de familie (HL7/FHIR).

1.2. Modele arhitecturale și tehnologii

Sistemul urmează un model arhitectural hibrid, adecvat naturii sale eterogene (edge + mobil + cloud + web):

- Three-tier (client-server, pe trei straturi)** pentru perimetrul Cloud + Web + persistență: stratul de prezentare (SPA React / UI mobil) → stratul de servicii (REST API Spring Boot) → stratul de date (RDS PostgreSQL). Aceasta este coloana vertebrală a soluției.
- Layered backend (Controller → Service → Repository):** controllerele expun REST, serviciile conțin logica de business și tranzacțiile, repository-urile (Spring Data JPA) accesează PostgreSQL.
- Store-and-forward** pentru fluxul dinspre senzori: ingestie idempotentă pe batchId, cozi locale offline-first pe mobil, alarme asincrone.
- Edge computing minimal pe ESP32:** doar achiziție, mediere pe fereastră și emitere BLE, fără logică decizională (decizia de alarmă rămâne pe mobil/Cloud).

Strat / componentă	Model arhitectural	Tehnologie (implementare)
--------------------	--------------------	---------------------------

ESP32	Microcontroller + peripheral BLE (GATT server)	C++ (Arduino framework, PlatformIO), BLE (BLEDevice), DHT, SEN-11574, AD8232
Android	MVVM + Coordinator, offline-first	Kotlin, Android SDK, Retrofit, BLE API (<i>nivel arhitectură</i>)
Cloud (backend)	Three-tier (Controller → Service → Repository)	Java 21, Spring Boot 3.3.4 (Web, Security, Data JPA, Validation, Actuator), JWT (jjwt 0.12.6), AWS Elastic Beanstalk (Docker)
Bază de date	Relațional normalizat (3FN)	PostgreSQL 18 (Amazon RDS)
Web (frontend)	SPA + client REST	React 18, react-router-dom, fetch API
Hosting web	CDN + object storage	Amazon CloudFront + S3
Integrare externă	Message/document-based	HL7 FHIR R4 (outbound, Bundle XML) generat în web; HL7 v2.x inbound (prevăzut în schema DB)

Componente refolosite (reuse): un contract comun de tipuri de date (DTO + enumerări) este partajat conceptual de toate modulele pentru consistență; pe partea Cloud se refolosesc starterele Spring Boot și mecanismele standard de autentificare JWT (bibliotecă jjwt); senzorii sunt module COTS (SEN-11574, AD8232, DHT); pe web se refolosesc utilitare comune (FhirXmlBuilder, dicționar ICD-10 local, componenta EcgMonitor).

1.3. Mediul de dezvoltare

Componentă	Mediu de dezvoltare (IDE)	Build / deploy
ESP32	PlatformIO (VS Code) / Arduino IDE	build firmware → flash USB pe modul
Android	Android Studio	Gradle → APK (<i>nivel arhitectură</i>)
Cloud backend	IntelliJ IDEA / VS Code	Maven (mvn clean package) → JAR → imagine Docker → Elastic Beanstalk
Bază de date	psql / pgAdmin / DBeaver	scripturi DDL (schema.sql) + seed (seed.sql)
Web	VS Code	npm run build → upload S3 → invalidare CloudFront
Control versiune	Git / GitHub	re-deploy manual (ZIP Docker în EB)

1.4. Sistemul de operare și mediul de execuție

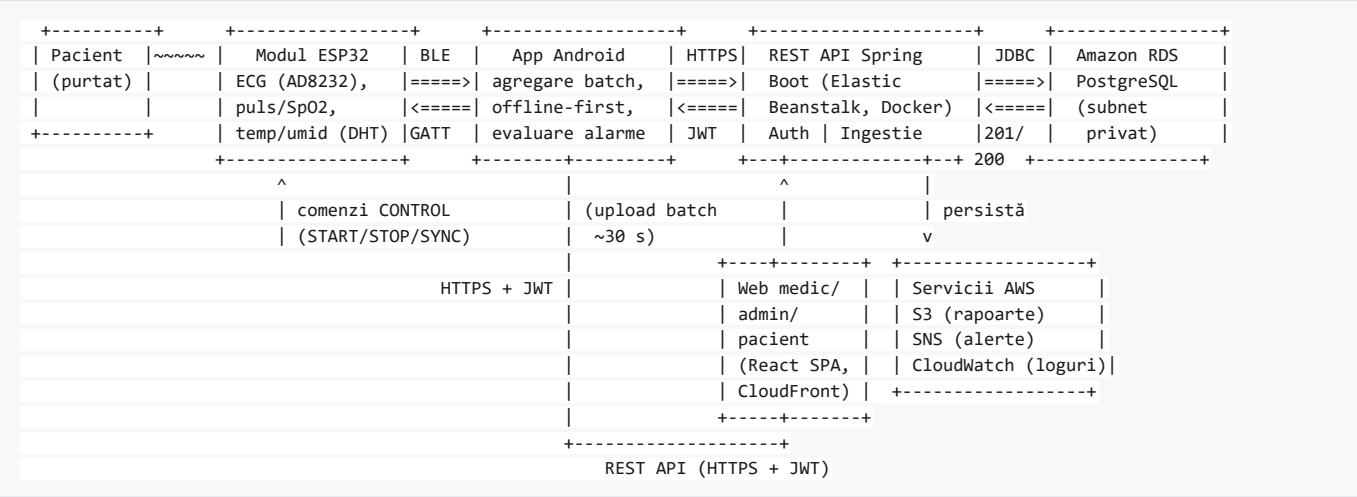
Întrucât sistemul este distribuit, mediul de execuție diferă pe componente:

Componentă	OS / runtime	Observații de proiectare
ESP32	Arduino framework pe ESP32 (FreeRTOS sub capotă)	bucă principală: achiziție @ ~1 s, mediere și NOTIFY BLE @ ~10 s, streaming ECG
Android	Android 10+ (API 29+)	client BLE + upload REST (<i>nivel arhitectură</i>)
Cloud	AWS Elastic Beanstalk - Docker pe Amazon Linux 2023, JVM 21	API REST + JWT; Tomcat (max 50 thread-uri); context-path /api; scalare configurabilă în EB
Bază de date	Amazon RDS PostgreSQL 18 (db.t3.micro, 20 GB)	în subnet privat (fără IP public); HikariCP (pool 2-10); dd1-auto: none (schema gestionată prin scripturi)
Web (hosting)	Amazon CloudFront + S3	conținut SPA servit prin CDN
Web (client)	browser-agnostic (Chrome/Edge/Firefox)	SPA responsive

Infrastructură Cloud (AWS, regiunea eu-central-1): VPC dedicat (subnets publice + private), **Elastic Beanstalk** pentru REST API (mediu seniorwatch-dev), **RDS PostgreSQL** pentru persistență (în subnet privat), **S3** pentru rapoarte PDF (reports, lifecycle 90 zile), **SNS** pentru notificări de alertă (email/push), **CloudFront + S3** pentru găzduirea aplicației web, **CloudWatch Logs** pentru observabilitate.

1.5. Schema-bloc a arhitecturii

Schema-bloc reflectă lanțul de participanți: **Pacient** → **Modul ESP32** → **App Android** → **REST API (Spring Boot/EB)** → **RDS PostgreSQL**, completat de aplicația Web (portal medic/pacient) și de sistemul extern al medicului de familie. Pacientul poartă modulul ESP32; datele curg dinspre senzori spre Cloud, iar răspunsurile (confirmări, recomandări) curg invers.



Interoperabilitate externă (HL7 FHIR R4):

- Portalul Web generează un **Bundle FHIR R4 (XML)** din datele pacientului, descărcat ca fișier `.xml` / transmis către sistemul medicului de familie.
- Schema bazei de date prevede tabelele `h17_inbound_referrals` (trimiteri) și `fhir_outbound_letters` (scrisori) pentru persistarea interoperabilității.

Lanțul principal de date:

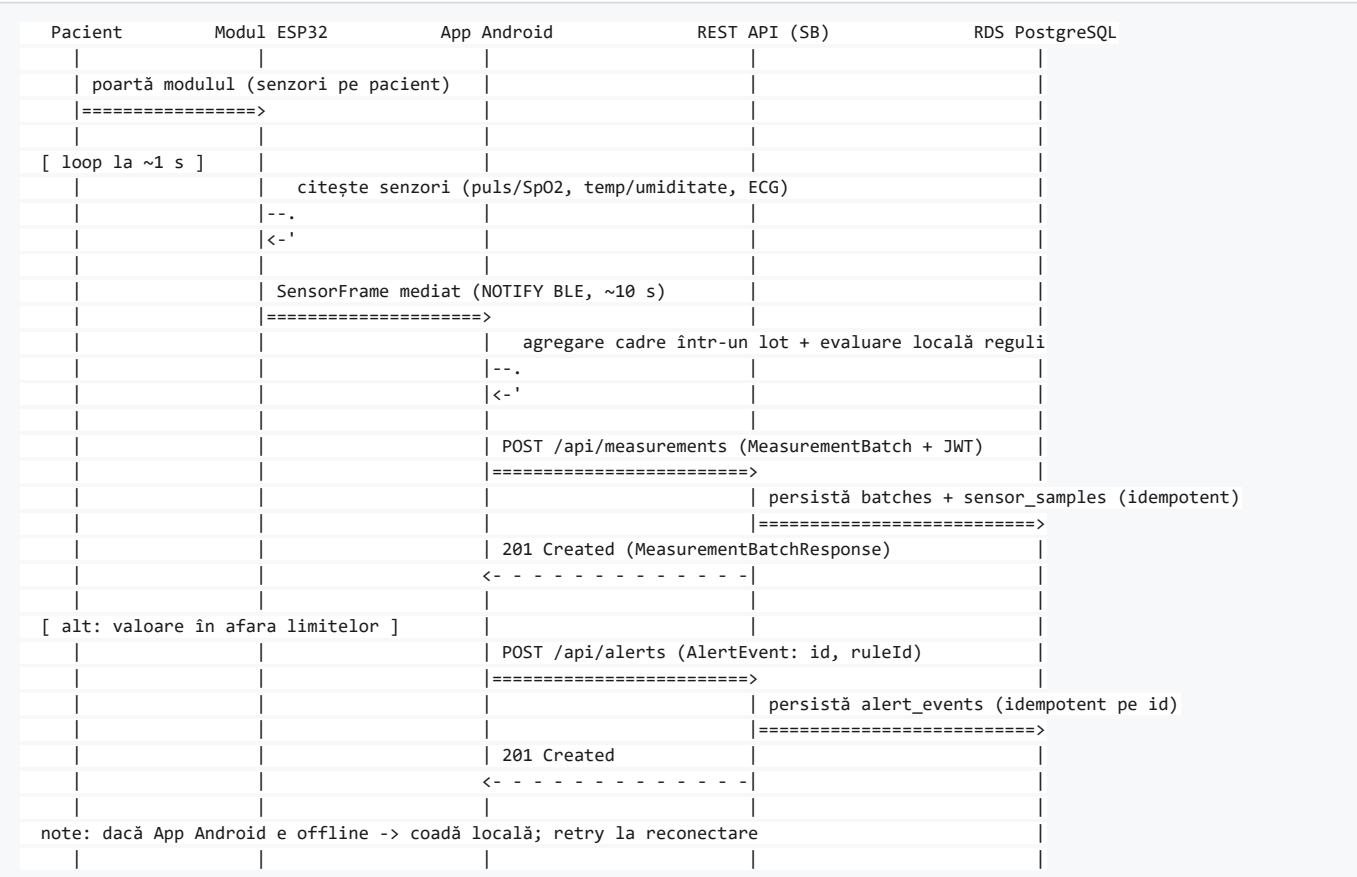
1. **Pacient → Modul ESP32:** pacientul poartă modulul; senzorii (AD8232 ECG, SEN-11574 puls, DHT temperatură/umiditate) furnizează semnalele.
2. **Modul ESP32 → App Android (BLE/GATT):** cadru `SensorFrame` mediat la ~10 s (puls, SpO₂, temperatură, umiditate, eșantioane ECG); flag `leadOff` la eveniment.
3. **App Android (local):** agregarea cadrelor într-un lot; evaluare locală a regulilor de alarmă.
4. **App Android → REST API (HTTPS + JWT):** POST `/api/measurements` (lot `MeasurementBatch`) → 201 Created; la valori în afara limitelor, POST `/api/alerts` (eveniment de alertă) → 201 Created.
5. **REST API → RDS PostgreSQL:** persistă `measurement_batches` (idempotent pe `batch_id`) + `sensor_samples`, respectiv `alert_events`.
6. **Offline:** dacă App Android este offline → coadă locală; retransmitere la reconectare.

Funcțiile pe componente:

Componentă	Funcții principale	Tehnologie
Modul ESP32	Achiziție senzori, mediere pe fereastră 10 s, server BLE GATT, streaming ECG, detecție lead-off	C++ (Arduino / PlatformIO)
App Android	Client BLE, agregare lot, evaluare alarme locale, upload REST, offline-first	Kotlin (<i>nivel arhitectură</i>)
REST API	API REST, autentificare/roluri (JWT), persistență, ingestie idempotentă, audit, expunere date pentru FHIR	Spring Boot 3.3.4 / Java 21 (Elastic Beanstalk)
RDS PostgreSQL	Persistența relațională a tuturor entităților (utilizatori, pacienți, măsurători, alarme, audit)	PostgreSQL 18 (Amazon RDS)
Web medic/admin/pacient	Fișă pacient, consultații ICD-10, alergii, recomandări, grafice, rapoarte, export FHIR	React SPA (CloudFront + S3)

1.6. Diagrama de secvență a fluxului principal

Diagrama de secvență reproduce fluxul principal (achiziție → agregare → ingestie → alarmă), cu participanții și etichetele corespunzătoare implementării: Pacient, Modul ESP32, App Android, REST API (Spring Boot), RDS PostgreSQL.



Legendă: SB = Spring Boot pe Elastic Beanstalk; => apel/răspuns sincron, - -> răspuns (return), --./<-' procesare locală (self-message).

1.7. Arborescența dialogurilor (forme aplicațiilor)

Aplicația este pe bază de dialog; se prezintă arborescența formelor pentru componentele cu interfață. Pentru aplicația web, arborescența corespunde rutelor reale definite în src/web/src/App.js.

Aplicația web - arborescența dialogurilor (rute reale):



└─ Alerte (/pacient/pacientalerte)

Aplicația mobilă Android (pacient) - arborescența dialogurilor (nivel arhitectură):

```
Login pacient
└─ Home
    ├── Status dispozitiv (conectat / deconectat / baterie)
    ├── Ultimele măsurători (puls, SpO2, temperatură)
    ├── Jurnal alarme locale
    ├── Recomandări primite
    └─ Setări (permisiuni BLE, sincronizare)
```

Elemente generice de interfață: ecrane responsive; protejarea rutelor pe roluri (ProtectedRoute cu allowedRoles); afișarea consistentă a alertelor (cod de culoare: roșu = critic, galben = avertizare, verde = normal); grafice de evoluție și monitor ECG; liste de mesaje de sistem pentru erori și confirmări.

2. Descrierea componentelor (modulelor)

Fiecare componentă de program este descrisă prin intrări, prelucrări, ieșiri și prin interacțiunile cu alte module. Pentru uniformitate se folosește un șablon comun.

2.1. Șablon unitar de descriere a modulelor

Denumire modul	: <nume tehnic>
Responsabilitate	: <o frază sintetică>
Tehnologie / runtime	: <limbaj + framework + OS>
Intrări	: <surse de date, tipuri DTO>
Prelucrări	: <algoritmi / reguli / transformări>
Ieșiri	: <destinații, tipuri DTO>
Interacțiuni externe	: <alte module / sisteme>
Responsabil	: <membru echipă>

2.2. Modulul ESP32 (src/embedded)

Atribut	Valoare
Responsabilitate	Achiziție semnale fiziologice și ambientale; mediere pe fereastră; emitere cadre BLE la ~10 s + streaming ECG
Tehnologie / runtime	C++ pe ESP32 (Arduino framework, PlatformIO), BLE GATT server (BLEDevice/BLEServer)
Intrări	Semnal analog ECG (AD8232, pin 34 + lead-off pe 26/25), puls (SEN-11574 pe pin analogic 32), temperatură/umiditate (DHT pe pin 27), comenzi BLE pe caracteristica CONTROL
Prelucrări	Citire la ~1 s; agregare/mediere pe fereastră de 10 s (SensorAggregator); compunerea structurii SensorFrame; detecție leadOff ECG
Ieșiri	SensorFrame prin NOTIFY pe caracteristica de TX; flux ECG (sendEcgStart/Value/End); nume advertising WH-0001
Interacțiuni externe	Aplicația Android (rol BLE central)
Responsabil	Mathe Alexandra

Sub-componente interne (clase firmware):

- BoardManager - inițializarea plăcii.
- PulseSensor - citire puls (PulseReading), detecție prezență deget (state machine).
- EnvSensor - citire DHT (EnvReading: temperatură, umiditate), interval minim de citire ~2 s.
- ECGSensor - citire ADC ECG (EcgSample: până la 50 valori brute / cadru, flag leadOff).
- SensorFrameBuilder - compune SensorFrame din cele trei surse (assembleTenSecondFrame).
- SensorAggregator - acumulează probe și calculează cadrul mediat (computeAverageFrame).
- BLEPeripheral - advertising, server GATT, notificări de cadru și flux ECG.
- PowerManager - gestiunea energiei.
- FirmwareMainLoop - orchestratorul buclei (init + loop la 1 s/10 s).

2.3. Modulul mobil Android (modul de arhitectură)

Descris pe baza specificației de arhitectură (architecture/module3-mobile). Codul sursă curent al aplicației mobile nu reflectă versiunea actuală a sistemului și nu a fost folosit ca sursă autoritară.

Atribut	Valoare
Responsabilitate	Client BLE; agregarea cadrelor într-un lot; upload batch la Cloud; alerte locale; offline-first
Tehnologie / runtime	Kotlin, Android SDK (API 29+), Retrofit, BLE API

Intrări	SensorFrame via BLE NOTIFY; recomandări descărcate de la Cloud
Prelucrări	(a) parsare cadru BLE → probă; (b) agregare într-un MeasurementBatch; (c) evaluare locală a regulilor pentru alarme rapide; (d) persistență offline + coadă de retrimiteri
Ieșiri	POST /api/measurements CU MeasurementBatch; POST /api/alerts CU AlertEvent; notificări UI
Interacțiuni externe	ESP32 (BLE), REST API (HTTPS REST + JWT)
Responsabil	Cotolan Denisa

2.4. Modulul Cloud (src/cloud/backend)

Atribut	Valoare
Responsabilitate	REST API central, autentificare, persistență, ingestie idempotentă, audit, expunere date pentru export FHIR
Tehnologie / runtime	Java 21, Spring Boot 3.3.4 (Web, Security, Data JPA, Validation, Actuator), jjwt 0.12.6, PostgreSQL driver; rulează pe AWS Elastic Beanstalk (Docker)
Intrări	Apeluri REST din web și mobil (JSON), cu Authorization: Bearer <JWT>
Prelucrări	Autorizare pe roluri (@PreAuthorize ADMIN/DOCTOR/PATIENT); validare DTO (jakarta.validation); deduplicare pe batchId/alertId; persistență JPA în PostgreSQL; jurnalizare audit; izolare per medic (doctor_id)
Ieșiri	Răspunsuri REST (DTO: MeasurementBatchResponse, PatientResponse, AlertResponse, EcgSeriesResponse etc.); LoginResponse cu token JWT
Interacțiuni externe	Web, Android; servicii AWS configurate (RDS, S3, SNS, CloudWatch)
Responsabil	Balan Dan (backend), Ardelean Marius (infra)

Controllere implementate (sub context-path /api):

Controller	Rute principale	Roluri
AuthController	POST /auth/login, POST /auth/reset-password	public
UserController	GET/POST /users, PUT /users/{id}, PATCH /users/{id}/active	ADMIN
PatientController	GET/POST /patients, GET /patients/{id}, GET /patients/me, PUT /patients/{id}	DOCTOR/ADMIN (+ PATIENT pe /me)
ClinicalVisitController	GET/POST /clinical-visits, PATCH /clinical-visits/{id}/finalize, GET /clinical-visits/patient/me	DOCTOR/ADMIN (+ PATIENT pe /me)
AllergyController	GET/POST /allergies/patient/{id}, GET /allergies/me, PUT /allergies/{id}, PUT /allergies/{id}/archive	DOCTOR/ADMIN (+ PATIENT pe /me)
RecommendationController	POST /recommendations, GET /recommendations/patient/{id}	DOCTOR/ADMIN (+ PATIENT)
MeasurementController	POST /measurements, GET /measurements/{patientId}, GET /measurements/{patientId}/ecg	autentificat
AlertController	POST /alerts, GET /alerts/{patientId}	autentificat
AuditController	GET /audit, POST /audit/export-report	ADMIN (+ DOCTOR pe export)

Arhitectura este pe straturi: **Controller** (REST) → **Service** (AuthService, PatientService, MeasurementService, AlertService, ClinicalVisitService, AllergyService, RecommendationService, UserService, AuditService) → **Repository** (Spring Data JPA) → **PostgreSQL**. Securitatea este stateless (SessionCreationPolicy.STATELESS), cu JwtFilter înaintea filtrului standard și BCryptPasswordEncoder pentru parole.

2.5. Modulul Web (src/web)

Atribut	Valoare
---------	---------

Responsabilitate	Portal medic / admin / pacient: fișă, consultații, alergii, recomandări, grafice, rapoarte, export HL7/FHIR
Tehnologie / runtime	React 18 (SPA), react-router-dom; client REST (fetch); hosting CloudFront + S3
Intrări	Input de la utilizator (formulare); date de la REST API (PatientResponse, ClinicalVisitResponse, AlertResponse, EcgSeriesResponse etc.); token JWT în localStorage (sw_token)
Prelucrări	Randarea fișei pacient; căutare ICD-10 (dicționar local icd10Codes.js); construirea graficelor; monitor ECG (EcgMonitor); generarea unui Bundle FHIR R4 XML (FhirXmlBuilder)
Ieșiri	CRUD asupra resurselor pacient; descărcare fișier FHIR R4 (.xml); jurnalizare export raport (/api/audit/export-report)
Interacțiuni externe	Cloud (REST + JWT)
Responsabil	Morgovan Raluca (medic), Nicola Larisa (pacient & rapoarte)

Module web notabile:

- `api.js` - centralizează apelurile REST (login, users, patients, clinical-visits, allergies, recommendations, measurements/ECG, alerts, audit) cu Authorization: Bearer.
- `ProtectedRoute.jsx` - protejarea rutelor pe roluri.
- `utils/fhirXmlBuilder.js` - construiește un **Bundle FHIR R4 (XML)** cu resurse Patient, Encounter, Condition, AllergyIntolerance, CarePlan, Flag, Observation (vital-signs LOINC).
- `components/EcgMonitor.jsx` - afișare formă de undă ECG (date reale din /api/measurements/{id}/ecg sau semnal simulat PQRST).
- `data/icd10Codes.js` - dicționar ICD-10 pentru codificarea diagnosticilor.

2.6. Componenta logică „Tipuri partajate” (shared)

Nu este un modul de execuție, ci un contract comun de date (DTO + enumerări) folosit pentru consistența intrărilor/ieșirilor între module:

- **identitate/auth:** LoginRequest, LoginResponse, ResetPasswordRequest, rol utilizator (ADMIN/DOCTOR/PATIENT);
- **pacient:** DemographicsDto, PatientRequest, PatientResponse;
- **clinic:** ClinicalVisitRequest/Response, AllergyRequest/Response, RecommendationRequest/Response;
- **senzori:** MeasurementBatchRequest, SensorSampleDto, MeasurementBatchResponse, EcgSeriesResponse;
- **alarmare:** AlertRequest, AlertResponse, enum AlertSeverity (WARNING/CRITICAL), enum SensorParameter;
- **versionare & audit:** VersionMetadata (coloane comune), enum HealthItemStatus, AuditResponse.

3. Descrierea comunicării între module

Fiecare canal de comunicație este descris prin modul de comunicare, lista de parametri, semnificațiile acestora și restricții. Se folosește un șablon comun.

3.1. Șablon unitar pentru descrierea canalelor

Canal	: <perimetru A ↔ B>
Suport fizic / transport	: <BLE / HTTPS>
Protocol	: <GATT / REST / FHIR R4>
Direcție	: <A → B / B → A / bidirecțional>
Autentificare	: <JWT / pairing BLE>
Format mesaj	: <binar packed / JSON / Bundle XML>
Frecvență	: <periodic / on-event>
Idempotență / retry	: <cheie + strategie>
Restricții	: <MTU, latență, dimensiuni payload>

3.2. Canalele efective din sistem

C1. ESP32 ↔ Android (BLE GATT)

Atribut	Valoare
Transport	Bluetooth Low Energy (GATT)
Direcție	ESP32 → Android (NOTIFY), Android → ESP32 (WRITE pe CONTROL)
Autentificare	Pairing BLE; corelarea sensorKitId cu device_bindings din Cloud
Format	Binar little-endian packed (SensorFrame); flux ECG separat (start/valori/end)
Frecvență	Cadru mediat la ~10 s; flag LEAD_OFF la eveniment
Restricții	MTU ~185 octeți; reconnect cu back-off exponențial (1→60 s)

Service UUID de referință: 0000FF01-0000-1000-8000-00805F9B34FB. Caracteristici GATT (de referință):

Caracteristică	UUID	Properties	Direcție	Semnificație
SENSOR_FRAME	0000FF02-...	NOTIFY	ESP32 → Android	cadru de măsurători mediat (10 s)
DEVICE_INFO	0000FF03-...	READ	ESP32 → Android	sensorKitId, versiune firmware, baterie
CONTROL	0000FF04-...	WRITE	Android → ESP32	comenzi (start/stop/led/ping/sync)
LEAD_OFF	0000FF05-...	NOTIFY	ESP32 → Android	flag electrozi ECG desprinși

Cadrul SensorFrame conține (conform firmware + specificație BLE): seqNumber, timestampMs, puls (heartRateBpm), SpO₂, umiditate, temperatură, eșantioane ECG (ADC pe 16 biți, până la 50-80 / cadru) și flags (bit0 = leadOff, bit1 = lowBattery). Comenzi CONTROL: 0x01 START_STREAM, 0x02 STOP_STREAM, 0x03 SET_LED, 0x04 PING, 0x05 SYNC_TIME.

C2. Android → Cloud (ingestie batch de măsurători)

Atribut	Valoare
Transport / Protocol	HTTPS / REST (Spring Boot pe Elastic Beanstalk)
Endpoint	POST /api/measurements
Autentificare	JWT Bearer (Authorization: Bearer <token>)

Corp request	MeasurementBatchRequest (JSON): batchId, patientId, deviceId, intervalStart, intervalEnd, samples[] (ts, puls, spo2, temperatura, umiditate, ecgBytes base64)
Corp response	MeasurementBatchResponse (JSON), 201 Created
Frecvență	periodic (~30 s, configurabil pe mobil)
Idempotență / retry	batchId unic (UUID generat de Android); re-trimiterile aceluiași batchId sunt ignorate → fără dubluri
Restricții	timeout client recomandat; payload JSON moderat

C3. Android → Cloud (alarmă)

Atribut	Valoare
Endpoint	POST /api/alerts
Corp request	AlertRequest (JSON): id (UUID client-side), patientId, ruleId, triggeredAt, severitate (WARNING/CRITICAL), textPacient
Corp response	AlertResponse, 201 Created
Prioritate	Asincron, imediat la declanșare; latență țintă < 10 s
Idempotență	AlertRequest.id (UUID): dacă există deja, se returnează evenimentul existent (fără duplicare). Opțional propagare prin SNS

C4. Cloud → Android / Pacient (downlink: recomandări)

Atribut	Valoare
Endpoint	GET /api/recommendations/patient/{patientId}
Autentificare	JWT Bearer
Corp response	RecommendationResponse[] (JSON)
Frecvență	la login + periodic

Regulile de alarmă (alarm_rules) sunt prevăzute în modelul de date pentru evaluarea pragurilor, dar nu sunt expuse printr-un endpoint REST dedicat în versiunea curentă.

C5. Web ↔ Cloud (CRUD + vizualizare)

Atribut	Valoare
Transport / Protocol	HTTPS / REST (JSON)
Autentificare	JWT Bearer (token în localStorage)
Exemple	POST /api/auth/login, GET /api/patients, GET /api/patients/{id}, POST /api/patients, POST /api/clinical-visits, GET /api/measurements/{patientId}, GET /api/measurements/{patientId}/ecg, GET /api/alerts/{patientId}, GET /api/users, POST /api/users, GET /api/audit
Format	JSON
Restricții	token expiră (implicit 3600 s) → 401 redirecționează la login; CORS configurat permisiv în SecurityConfig

C6. Cloud ← Sistem Medic de Familie (HL7 inbound - referral) (prevăzut în model)

Atribut	Valoare
Stocare	tabela h17_inbound_referrals (mesaj integral + status RECEIVED/PROCESSED/ERROR)
Stadiu	model de date pregătit; fără controller REST dedicat în versiunea curentă

C7. Web → Sistem Medic de Familie (HL7 FHIR R4 outbound - scrisoare/export)

Atribut	Valoare
Generare	în portalul web (FhirXmlBuilder), pe baza datelor obținute prin REST
Format	Bundle FHIR R4 (XML): Patient, Encounter, Condition (ICD-10), AllergyIntolerance, CarePlan, Flag, Observation (vital-signs LOINC)
Acțiune	descărcare fișier .xml (application/fhir+xml) sau copiere; transmitere către sistemul MF
Stocare	tabela fhir_outbound_letters prevăzută pentru persistarea scrisorilor (bundle_json + status)

3.3. Harta integrată a canalelor

```

[ESP32] --(C1 BLE/GATT, cadru ~10 s)--> [Android]
  ^
  |
  |<----- comenzi CONTROL -----|

[Android] --(C2 HTTPS POST /api/measurements, ~30 s)--> [REST API Spring Boot]
[Android] --(C3 HTTPS POST /api/alerts, asincron)-----> [REST API Spring Boot]
[REST API] --(C4 HTTPS GET /api/recommendations)-----> [Android / Pacient]

[Web] <----(C5 HTTPS REST CRUD + grafice)----> [REST API Spring Boot]

[Web] --(C7 FHIR R4 Bundle XML, generat client-side)--> [fișier .xml / Sistem MF]
[Sistem MF] --(C6 HL7 inbound, prevăzut în model)-----> [REST API / DB]

```

Mecanisme transversale de comunicare: idempotență (batchId / alertId), offline-first (coadă locală + retransmitere la reconectare), securitate TLS + JWT, observabilitate (CloudWatch Logs), notificări prin Amazon SNS la alarme.

4. Structuri de baze de date și fișiere

Baza de date este relațională, **PostgreSQL 18** (Amazon RDS), cu **21 de tabele**, definită în `src/cloud/database/schema.sql`. Tipurile de date sunt specifice PostgreSQL (UUID, VARCHAR, CHAR, TEXT, TIMESTAMPTZ, DATE, NUMERIC, SMALLINT, INT, BIGINT, BOOLEAN, BYTEA). PostgreSQL dispune de tipuri enumerate native, folosite pentru domeniile de valori.

Domenii de valori (tipuri ENUM native + CHECK):

Domeniu	Implementare	Valori
user_role	ENUM	ADMIN, DOCTOR, PATIENT
health_item_status	ENUM	ACTIVE, ARCHIVED, DELETED, AMENDED
alert_severity	ENUM	WARNING, CRITICAL
sensor_parameter	ENUM	ECG, UMIDITATE, TEMPERATURA, PULS
permissions.action	CHECK	READ, WRITE, DELETE, EXPORT
allergies.severity	CHECK	MILD, MODERATE, SEVERE
audit_events.event_type	CHECK	LOGIN, LOGOUT, READ, CREATE, UPDATE, DELETE, EXPORT, IMPORT

4.1. Schema generală - diagrama ER



Legături suplimentare (măsurători, alerte, interoperabilitate):

Entitate 1	Relație	Entitate 2	Observații
patients	1:N	measurement_batches	ingestie loturi senzori
measurement_batches	1:N	sensor_samples	eșantioane per lot
measurement_batches	1:N	accel_bursts	burst accelerometru
accel_bursts	1:N	accel_samples	eșantioane accel.
patients	1:N	alert_events	evenimente de alarmă
alert_events	N:1	alarm_rules	regula declanșată

patients	1:N	hl7_inbound_referrals	trimiteri HL7 inbound
patients	1:N	fhir_outbound_letters	scrisori FHIR outbound
fhir_outbound_letters	N:1	clinical_visits	consultație asociată (opțional)
users	1:N	audit_events	jurnal append-only

4.2. Definiții de câmp pe tabele

Pentru fiecare câmp se prezintă: tip, restricții și semnificația. Coloanele de versionare (VersionMetadata) sunt comune entităților „health item” și se descriu o singură dată (§4.2.13).

4.2.1. users

Câmp	Tip	Restricții	Semnificație
id	UUID	PK, default gen_random_uuid()	identificator utilizator
email	VARCHAR(120)	NOT NULL, UNIQUE	identificare login
password_hash	VARCHAR(255)	NOT NULL	parolă hash bcrypt
role	user_role	NOT NULL	rol (ADMIN/DOCTOR/PATIENT)
active	BOOLEAN	NOT NULL, default TRUE	cont activ
created_at / updated_at	TIMESTAMPZ	NOT NULL, default NOW()	auditabil

4.2.2. permissions

Câmp	Tip	Restricții	Semnificație
id	UUID	PK	identificator
role	user_role	NOT NULL	rolul vizat
resource	VARCHAR(80)	NOT NULL	resursa protejată
action	VARCHAR(20)	CHECK IN (READ,WRITE,DELETE,EXPORT)	acțiune permisă
confidential	BOOLEAN	NOT NULL, default FALSE	marcaj confidențialitate
-	-	UNIQUE (role, resource, action)	unicitate triplet

4.2.3. patients

Câmp	Tip	Restricții	Semnificație
id	UUID	PK	identificator pacient
doctor_id	UUID	FK → users.id, NOT NULL, INDEX	medic asignat (izolare date)
active	BOOLEAN	NOT NULL, default TRUE	status logic
created_at	TIMESTAMPZ	NOT NULL, default NOW()	auditabil

4.2.4. demographics (1:1 cu patients)

Câmp	Tip	Restricții	Semnificație
patient_id	UUID	PK, FK → patients.id ON DELETE CASCADE	legătură 1:1
nume / prenume	VARCHAR(80)	NOT NULL	identificare pacient
sex	CHAR(1)	CHECK IN ('M','F','O')	sex
data_nasterii	DATE	NOT NULL	dată naștere

cnp	CHAR(13)	UNIQUE, INDEX	identificator național
strada,localitate,judet	VARCHAR(80)	-	adresă structurată
cod_postal	VARCHAR(10)	-	cod poștal
tara	VARCHAR(80)	default 'Romania'	țară
telefon	VARCHAR(20)	-	contact
email	VARCHAR(120)	INDEX	contact
profesie,loc_de_munca	VARCHAR(120)	-	date socio-profesionale

4.2.5. medical_records (1:1, versionat)

Câmp	Tip	Restricții	Semnificație
patient_id	UUID	PK, FK → patients.id ON DELETE CASCADE	legătură 1:1
istoric_medical	TEXT	-	istoric
alergii_text	TEXT	-	alergii (text liber)
consultatii_cardio_text	TEXT	-	note cardiologice
(+ VersionMetadata - §4.2.13)			versionare append-only

4.2.6. device_bindings (asociere unică pacient ↔ smartphone ↔ senzori)

Câmp	Tip	Restricții	Semnificație
id	UUID	PK	identificator
patient_id	UUID	UNIQUE, FK → patients.id	un binding per pacient
smartphone_device_id	VARCHAR(128)	-	identificator telefon
sensor_kit_id	VARCHAR(128)	NOT NULL, UNIQUE	identificator set senzori (corelat BLE)
created_at	TIMESTAMPZ	NOT NULL, default NOW()	auditabil

4.2.7. alarm_rules (reguli de alarmă, versionat)

Câmp	Tip	Restricții	Semnificație
id	UUID	PK	identificator regulă
patient_id	UUID	FK, NOT NULL, INDEX	pacient vizat
parametru	sensor_parameter	NOT NULL	parametru vizat
prag_min	NUMERIC(10,2)	NOT NULL	prag minim
prag_max	NUMERIC(10,2)	NOT NULL, CHECK prag_max > prag_min	prag maxim
durata_persistenta_sec	INT	NOT NULL default 0, CHECK ≥ 0	durată de persistență
interval_debut_activitate_sec	INT	NOT NULL default 0, CHECK ≥ 0	interval de la debutul activității
(+ VersionMetadata - §4.2.13)			versionare

4.2.8. normal_range_profiles (profil valori normale, 1:1)

Câmp	Tip	Restricții	Semnificație
id	UUID	PK	identificator

patient_id	UUID	UNIQUE, FK, NOT NULL	un profil per pacient
note	TEXT	-	observații
created_at	TIMESTAMPZ	NOT NULL, default NOW()	auditabil

4.2.9. clinical_visits (consultații ICD-10, versionat)

Câmp	Tip	Restricții	Semnificație
id	UUID	PK	identificator consultație
patient_id	UUID	FK, NOT NULL, INDEX	pacient
visited_at	TIMESTAMPZ	NOT NULL, INDEX	moment consultație
motiv_prezentare	VARCHAR(500)	-	motivul prezentării
simptome	TEXT	-	simptome
diagnostic_icd10_code	VARCHAR(10)	INDEX	cod ICD-10
diagnostic_icd10_display	VARCHAR(200)	-	descriere ICD-10
trimiteri	TEXT	-	trimiteri
retete	TEXT	-	rețete
(+ VersionMetadata - §4.2.13)			versionare

4.2.10. health_problems / allergies / medications (versionate)

- **health_problems:** id (UUID, PK), patient_id (FK, INDEX), description (TEXT, NOT NULL), icd10_code (VARCHAR 10), icd10_display (VARCHAR 200), onset_date (DATE) + VersionMetadata.
- **allergies:** id (UUID, PK), patient_id (FK, INDEX), substance_code (VARCHAR 50), substance_display (VARCHAR 200, NOT NULL), reaction (VARCHAR 200), severity (VARCHAR 20, CHECK IN MILD/MODERATE/SEVERE) + VersionMetadata.
- **medications:** id (UUID, PK), patient_id (FK, INDEX), product_code (VARCHAR 50), product_display (VARCHAR 200, NOT NULL), dosage (VARCHAR 100), frequency (VARCHAR 100), prescribed_at (TIMESTAMPZ, default NOW()) + VersionMetadata.

4.2.11. recommendations (recomandări de activitate, versionat)

Câmp	Tip	Restricții	Semnificație
id	UUID	PK	identificator
patient_id	UUID	FK, NOT NULL, INDEX	pacient
tip_activitate	VARCHAR(120)	-	tip activitate
durata_zilnica_minute	INT	-	durată zilnică
alte_indicatii	TEXT	-	indicații suplimentare
(+ VersionMetadata - §4.2.13)			versionare

4.2.12. Măsurători și alarme

measurement_batches (cheie de idempotență):

Câmp	Tip	Restricții	Semnificație
batch_id	VARCHAR(64)	PK	cheie de idempotență (UUID Android)
patient_id	UUID	FK, NOT NULL, INDEX (cu interval_start DESC)	pacient
device_id	VARCHAR(128)	FK → device_bindings.sensor_kit_id	dispozitiv

interval_start / interval_end	TIMESTAMPZ	NOT NULL	fereastra agregată
received_at	TIMESTAMPZ	NOT NULL, default NOW()	moment recepție

sensor_samples: id (BIGINT IDENTITY, PK), batch_id (FK → measurement_batches), ts (TIMESTAMPZ, NOT NULL, INDEX cu batch), puls (SMALLINT, CHECK 0-300), spo2 (SMALLINT, CHECK 0-100), temperatura (NUMERIC(4,1)), umiditate (NUMERIC(4,1)), ecg_blob (BYTEA, opțional - eșantioane ECG codate).

accel_bursts: id (BIGINT IDENTITY, PK), batch_id (FK), interval_start, interval_end (TIMESTAMPZ). **accel_samples:** id (BIGINT IDENTITY, PK), burst_id (FK → accel_bursts.id), t (TIMESTAMPZ), x,y,z (NUMERIC(8,4)). *(Tabele prevăzute pentru date de accelerometru; nu sunt populate prin endpoint-ul de ingestie din versiunea curentă.)*

alert_events: id (UUID, PK - generat client-side pentru idempotență), patient_id (FK, NOT NULL, INDEX cu triggered_at DESC), rule_id (FK → alarm_rules.id, NOT NULL), triggered_at (TIMESTAMPZ, NOT NULL), severitate (alert_severity, NOT NULL), text_pacient (VARCHAR 500), received_at (TIMESTAMPZ, default NOW()).

4.2.13. Coloanele de versionare (VersionMetadata) - comune entităților clinice

Aplicate pe `medical_records`, `alarm_rules`, `clinical_visits`, `health_problems`, `allergies`, `medications`, `recommendations`:

Câmp	Tip	Restricții	Semnificație
version_id	UUID	NOT NULL, UNIQUE, default gen_random_uuid()	id versiune curentă
health_item_id	UUID	NOT NULL, default gen_random_uuid()	id logic stabil al elementului
recorded_at	TIMESTAMPZ	NOT NULL, default NOW()	moment înregistrare
recorded_by_user_id	UUID	FK → users.id	autor
responsible_person_id	UUID	FK → users.id	responsabil
status	health_item_status	NOT NULL, default 'ACTIVE'	stare element
previous_version_id	UUID	-	înlănțuire versiuni (append-only)

4.2.14. Audit și interoperabilitate

• **audit_events** (append-only, EuroRec 26/28): id (BIGINT IDENTITY, PK), occurred_at (TIMESTAMPZ, INDEX, default NOW()), user_id (FK), event_type (VARCHAR 32, CHECK IN LOGIN/LOGOUT/READ/CREATE/UPDATE/DELETE/EXPORT/IMPORT), resource (VARCHAR 120), resource_id (UUID), client_ip (VARCHAR 45), outcome (VARCHAR 16, CHECK IN SUCCESS/DENIED).

• **h17_inbound_referrals:** id (UUID, PK), raw_message (TEXT, NOT NULL), received_at (TIMESTAMPZ, default NOW()), patient_id (FK), status (VARCHAR 20, CHECK IN RECEIVED/PROCESSED/ERROR), error_msg (TEXT).

• **fhir_outbound_letters:** id (UUID, PK), patient_id (FK, NOT NULL), visit_id (FK → clinical_visits.id), bundle_json (TEXT, NOT NULL), sent_at (TIMESTAMPZ), status (VARCHAR 20, CHECK IN PENDING/SENT/ERROR), error_msg (TEXT).

4.3. Legături, indecși și drepturi de acces

Legături principale (chei externe):

- `demographics.patient_id` → `patients.id` (1:1, CASCADE)
- `medical_records.patient_id` → `patients.id` (1:1, CASCADE)
- `device_bindings.patient_id` → `patients.id` (1:1)
- `clinical_visits` / `health_problems` / `allergies` / `medications` / `recommendations` / `alarm_rules` . `patient_id` → `patients.id` (1:N)
- `sensor_samples.batch_id` → `measurement_batches.batch_id` (1:N)
- `accel_samples.burst_id` → `accel_bursts.id` (1:N); `accel_bursts.batch_id` → `measurement_batches.batch_id`
- `alert_events.patient_id` → `patients.id` (1:N); `alert_events.rule_id` → `alarm_rules.id` (N:1)
- `fhir_outbound_letters.visit_id` → `clinical_visits.id` (N:1)
- `patients.doctor_id` → `users.id` (N:1); `audit_events.user_id` → `users.id` (N:1)

Indecși (definiți în `schema.sql`):

Tabel	Index	Motivație
patients	idx_patients_doctor (doctor_id)	filtrare per medic
demographics	idx_demographics_cnp, idx_demographics_email	identificare rapidă
clinical_visits	(patient_id, visited_at DESC), (diagnostic_icd10_code)	fișă consolidată + căutare ICD-10
alarm_rules	idx_alarm_rules_patient (patient_id)	reguli per pacient
health_problems / allergies / medications / recommendations	(patient_id)	listare per pacient
measurement_batches	(patient_id, interval_start DESC)	interogări pe timp
sensor_samples	idx_sensor_samples_batch_ts (batch_id, ts)	serii temporale / grafice
alert_events	(patient_id, triggered_at DESC)	„ultimele alarme”
audit_events	(occurred_at DESC), (user_id, occurred_at DESC)	rapoarte de audit

Drepturi de acces (rol DB + autorizare API):

Rol DB / aplicație	Domeniu	Operațiuni
sw_app (rol DB)	toate tabelele	acces aplicație (Spring Boot); postgres doar pentru migrări/setup
ADMIN (rol app)	utilizatori, roluri, audit	gestiune completă (@PreAuthorize("hasRole('ADMIN')"))
DOCTOR (rol app)	pacienți/consultații proprii	filtrat pe doctor_id = @me
PATIENT (rol app)	propriile date (/me, recomandări, măsurători, alarme)	SELECT pe patient_id = @me
ingestie (token mobil)	measurement_batches, sensor_samples, alert_events	INSERT (idempotent)

Permisiunile sunt încărcate inițial (seed) în tabela `permissions`: DOCTOR (acces complet la datele pacienților proprii), PATIENT (citire măsurători/recomandări/alarmele), ADMIN (utilizatori, permisiuni, audit).

4.4. Normalizarea

- Schema este normalizată până la a 3-a formă normală (3FN):
- 1FN/2FN:** atributele non-cheie depind exclusiv de cheia primară; relațiile 1:1 (`demographics`, `medical_records`, `device_bindings`, `normal_range_profiles`) izolează grupuri funcționale de atribute ale pacientului.
- 3FN:** nu există dependențe tranzitive - adresa face parte din `demographics` pentru că descrie direct pacientul.
- Denormalizări controlate (documentate):**
 - valorile codificate ICD-10 (*_code + *_display) sunt păstrate pe rândul clinic (`clinical_visits`, `health_problems`) din motive de istoric clinic imuabil;
 - `sensor_samples` este menținut „flat” (fără tabele de dimensiune) pentru performanță la interogările de serii temporale.
 - Versionare append-only** (nu UPDATE in-place) pe entitățile clinice, prin coloanele `VersionMetadata` - conformă EuroRec Seal L2 și trasabilității GDPR.

4.5. Fișiere și stocare în afara bazei de date

Conținut	Tip stocare	Format
bundle FHIR R4 (export pacienți)	generat în web, descărcat ca fișier	XML (<code>application/fhir+xml</code>)
raw_message HL7 inbound	coloană TEXT în <code>h17_inbound_referrals</code>	text HL7
bundle_json FHIR outbound	coloană TEXT în <code>fhir_outbound_letters</code>	JSON/XML
Rapoarte generate (PDF/CSV)	Amazon S3 (bucket reports, lifecycle 90 zile)	PDF / CSV

Notificări de alertă	Amazon SNS	mesaj push/email
Log-uri tehnice / observabilitate	Amazon CloudWatch Logs	structurat

Anexă A. Trasabilitate Specificații → Proiectare

Cerință din Specificații	Acoperire în proiectare
F1 - Monitorizarea parametrilor	§2.2 (ESP32) + §3.2/C1 + §4.2.12 (sensor_samples)
F2 - Gestionarea alertelor < 10 s	§2.3 (mobil) + §2.4 (AlertController/AlertService) + §3.2/C3 + §4.2.12 (alert_events) + §1.6
F3 - Administrarea fișei pacientului (ICD-10)	§2.4 (ClinicalVisitController) + §2.5 (Web) + §4.2.9 (clinical_visits)
F4 - Valori normale & reguli de alarmă	§4.2.7 (alarm_rules) + §4.2.8 (normal_range_profiles)
F5 - Recomandări de activitate	§2.4 (RecommendationController) + §3.2/C4 + §4.2.11 (recommendations)
F6 - Comunicare HL7 / FHIR	§2.5 (FhirXmlBuilder) + §3.2/C6-C7 + §4.2.14 (hl7_inbound_referrals, fhir_outbound_letters)
F7 - Raportare și vizualizare	§1.7 (forme) + §2.5 (EcgMonitor, rapoarte) + §4.5 (S3)
F8 - Securitate, roluri, audit	§2.4 (SecurityConfig/JwtUtil/JWT) + §4.3 (roluri) + §4.2.14 (audit_events)
Offline + sincronizare	§2.3 (mobil) + §3.2/C2 (idempotență pe batchId)
Restricție ESP32 - Bluetooth exclusiv	§3.2/C1